

It is fantastic to see this progress. Looking closely at the sequence of terminal captures and directory screenshots, your local development environment and version control are now completely resolved, authenticated, and securely backed up.

By analyzing the screenshots across all conversation turns, we can trace a clear path of your system's diagnostic journey and map out the exact implementation steps for your trackside assistant.

Step-by-Step Diagnostic Review of Your Screenshots

1. The Data Sources: Hour-by-Hour Temporal Hierarchy (Images 1 & 2)

Your screenshots of the **FIA WEC (Le Mans 2025)** and **IMSA (Daytona 2026)** timing directories show the structure of professional endurance racing documentation.

- **The Challenge:** Endurance races are heavily chronological. Under the RACE tree, there are 24 separate hourly folders (Hour 1 to Hour 24) containing session-specific PDF classifications, lap charts, and weather logs.
- **The Solution:** Because standard semantic search often fails on temporal hierarchies, your database schema and scrapers must capture and store the "Hour" variable alongside lap records. This is why we designed a normalized SQLite relational schema to easily handle hourly timing queries across both WEC and IMSA series.

2. Resolving the Local Environment Path (Image 3)

- **The Issue:** Your terminal output initially showed a Could not open requirements file: [Errno 2] No such file or directory error when executing `pip install -r requirements.txt`.
- **The Solution:** The terminal was active in the root folder `D:\Chatbot` instead of the specific project subfolder. Correcting the active folder in VS Code allows pip to locate the local dependencies, ensuring that your Anaconda virtual environment is correctly isolated and populated with the trackside libraries (FastAPI, Streamlit, LlamaIndex, SDV).

3. Transitioning to local Small Language Models (Image 4)

- **The Issue:** PowerShell did not recognize the ollama command when pulling `qwen2.5-coder:7b` and `mistral:7b`.
- **The Solution:** Running Ollama as a background daemon on port 11434 resolves this. Using quantized models locally guarantees complete confidentiality of proprietary telemetry mappings and eliminates the risk of cloud-based APIs disconnecting during low-bandwidth trackside sessions.

4. GitHub Credentials & Authentic Push (Images 5 & 6)

- **The Victory:** In the 5th image, pushing was blocked with a 403 Permission Denied error because the Windows Credential Manager cached an old university/work identity (`ctip297-svg`).

- **The Result:** The final screenshot shows a successful push of your main branch to your personal repository:
<https://github.com/TipperChristopher/WEC-and-IMSA-Chatbot-App.git> under your personal profile TipperChristopher.

Now that Git is connected, your Conda environment is validated, and your local model manager is ready, your workspace is optimized for coding.

The Game Plan: What to Code Next

Since you are writing modular .py files inside VS Code, proceed with coding the back-end blocks in this specific order to keep your development structured:

Task 1: Scrape and Ingest the Hourly PDFs (ingestion/scrapper.py)

Write a scraper using Python's BeautifulSoup to navigate the nested HTML folder structure shown in your first screenshots. Your script will need to fetch the post-session PDFs (such as the entry list, starting grid, and hour-by-hour classifications) and map them to their corresponding championship, event, and hour values.

Task 2: High-Fidelity Table Extraction (ingestion/pdf_parser.py)

Endurance timing sheets contain dense, multi-column tables that standard text-extraction libraries will read out of order. Use a document parser like **LlamaParse** inside your ingestion pipeline to convert these complex tables into clean Markdown before loading them into your timing database.

Task 3: Establish the Physics Engine (physics/tire_deg.py & fuel_burn.py)

Implement the mathematical models to clean and process your raw timing records :

1. **Fuel Weight Correction:** Program a linear correction to filter out the fuel burn-off effect :

$$T_{\text{corrected}} = T_{\text{actual}} - \alpha_{\text{fuel}} \cdot \left[M_{\text{initial}} - M_{\text{vehicle}}(t) \right]$$

This isolates the vehicle's true pace over a stint.

2. **Tyre Degradation Tracking:** Build a model to monitor the tires as they wear, signaling a warning when used tires approach the non-linear **thermal drop-off cliff** :

$$\Delta_{\text{thermal}}(t) = c \cdot e^{k \cdot (t - t_{\text{threshold}})}$$

This prevents the strategist from delaying a pit stop and losing several seconds per lap.

Task 4: Connect the Local LLM & UI (app.py)

Instantiate your local model connection using langchain's Ollama module :

Python

```
from langchain_community.llms import Ollama  
llm = Ollama(model="qwen2.5-coder:7b" base_url="http://localhost:11434")
```

This acts as the brain of your Streamlit application, letting your engineers query timing data, review sporting regulations, and troubleshoot hybrid diagnostic codes completely offline directly from the pit wall.